Sensitivity Is Not Fragility: Shared Visual Responses and Task Outcomes in Vision-Language-Action Policies

Anonymous CVPR submission

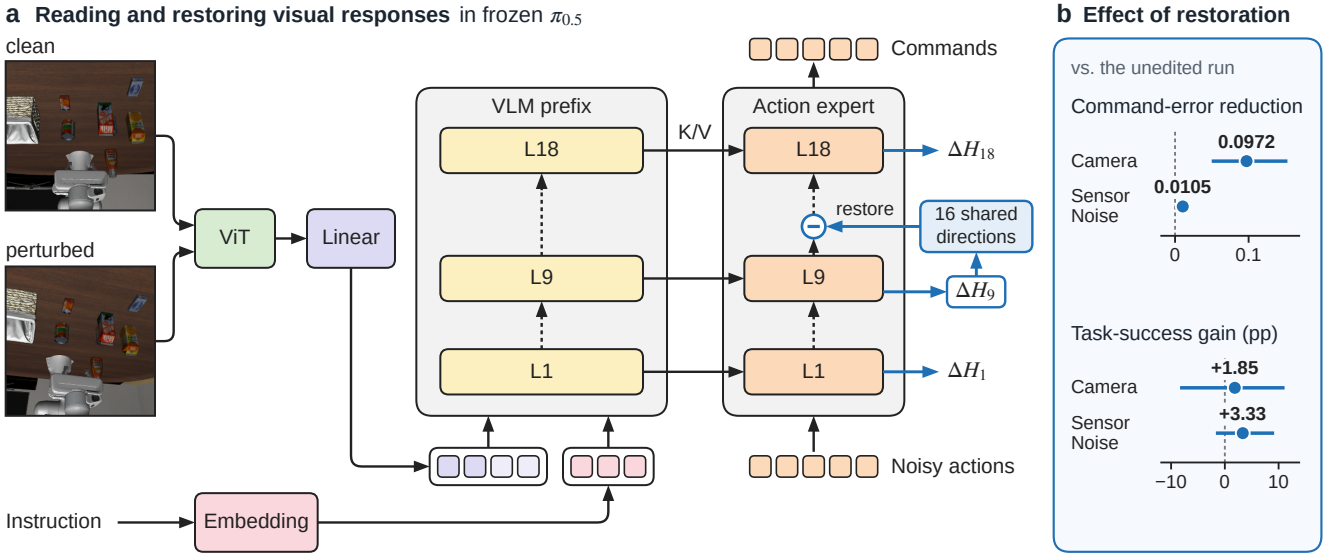


Figure 1. **Where a perturbation's effect is recorded and restored in frozen $\pi_{0.5}$.** (a) A clean and a perturbed observation of the same physical state (main camera shown) pass through the policy with the same policy noise. Each block of the action expert attends to the image and instruction tokens (K/V) while it turns noisy actions into commands. After action-expert blocks L1, L9, and L18 we record the response, the perturbed-minus-clean feature difference $\Delta H_\ell = H_\ell^p - H_\ell^c$, and fit 16 shared directions per layer (Fig. 3). Restoration copies the clean values along these directions back after L9. (b) Restoration compared with the unedited perturbed run (means, simultaneous 95% intervals; Table 2). The error of the first five commands falls by 0.0972 (24%) under Camera and 0.0105 (5%) under Sensor Noise; task success is higher by 1.85 (Camera) and 3.33 (Sensor Noise) percentage points.

Abstract

Visual perturbations that leave a task unchanged can sharply reduce the success of vision-language-action (VLA) policies. Making these policies robust requires knowing what a perturbation changes inside them and how to judge a correction. How much a perturbation changes features, commands, or motion (its sensitivity*) is the natural measure, yet it need not tell how much success the perturbation costs (its* fragility*). We trace perturbations through frozen $\pi_{0.5}$ and π_0 on LIBERO and LIBERO-Plus, from features to commands, motion, and task outcomes. Inside $\pi_{0.5}$, different perturbations change the features along 16 shared directions per layer: on held-out tasks and perturbation families, these capture 36.5–83.6% of the variation in the changes, against at most 10.2% for other sets of 16 directions. They give one common place to intervene. Copying clean values back along them, using a clean observation of the same state, reduces the error of the first five commands by 24% under camera changes and 5% under sensor noise, and task success is higher as well. These policies generate actions from random noise, so two unperturbed runs already move differently, which gives a yardstick: the clean rerun. Perturbation families that move the robot more tend to cost more success, but within a family the size of the early motion change barely tells which runs fail. Under sensor noise, π_0 deviates less from its clean run than a clean rerun does, and a rerun never changes the outcome, yet the perturbation costs 14.3 points of success. Net success also hides lost and gained successes. Robustness, and any internal correction, should therefore be judged by paired task outcomes against clean reruns.*

1. Introduction

Vision-language-action (VLA) models generate robot actions from images and a language instruction [2, 3, 11, 17]. Yet visual changes that leave the task unchanged can sharply reduce their success: LIBERO-Plus reports that π_0 falls from 94.2% on LIBERO to 13.8% under camera perturbations [5]. Such failures are naturally studied through how much a perturbation changes the policy's features, commands, or motion (its *sensitivity*); the task success it costs is its *fragility*. We show that the two are different things.

The policy's own randomness exposes the gap. VLAs such as π_0 act in a closed loop: from each observation they generate a sequence of future actions, execute the first five as controls, and replan, dozens of times per episode. Each sequence starts from a random sample, the *policy noise*, so two unperturbed runs from the same state already move differently. Under sensor noise in the images, π_0's end effector is, after the first five controls, 3.5 mm from where the clean run put it, while a second clean run lands 6.1 mm away. Yet a second clean run never changes the task outcome here, and the sensor noise costs 14.3 points of success. The converse also occurs: removing a headlight changes $\pi_{0.5}$'s first five commands 5.1 times as much as a dimming matched in image statistics, and both succeed in 77 of 80 runs.

To locate the gap, we follow a perturbation through the policy: image → features → commands → motion → outcome. A clean and a perturbed run that start with the same policy noise see the same physical state only at the episode's first policy call; afterwards the two robots move apart. We therefore compare the features and the first five commands sent to the robot at that call, motion after 5 controls (before any replanning) and after 60 (eleven replans later), and success at the end of the episode (Fig. 2). We ask three questions along this path:

1. *Image to features:* do different perturbations change the features along shared directions?
2. *Features to commands:* does restoring clean values along them move the commands toward the clean run's?
3. *Commands to motion and outcome:* how do command changes relate to motion and success?

Shared responses would give a concrete way to study visual robustness: different disturbances could be examined through common internal components, and those components could be tested directly during action generation.

We study frozen, LIBERO-finetuned $\pi_{0.5}$ [17] and π_0 [2] on LIBERO [12] and LIBERO-Plus under four perturbation families, Background, Camera, Light, and Sensor Noise, in about 4,000 simulator rollouts (Table 1), with a further 6,048 in a supplementary study (Sec. 4.2). We call the change a perturbation causes in the features its *response*. Our main findings are:

Different perturbations change the features along shared directions. In $\pi_{0.5}$, 16 directions per layer, fitted to responses, retain 36.5–83.6% of the response energy on held-out tasks and perturbation families, against at most 10.2% for 16 directions fitted to clean features or drawn at random (Figs. 1 and 3; Sec. 4.1).

Restoring clean values along these directions reduces command error. As a causal test, we copy clean values back along the shared directions at a middle layer (L9), given a clean observation of the same state. On held-out tasks this *restoration* moves the first five commands toward the clean run's, reducing their error by 24% under Camera and 5% under Sensor Noise. The 16 directions achieve most of what replacing the whole layer does. Task success is also higher, by 1.85 and 3.33 points (Figs. 1b and 4; Sec. 4.2).

Command changes reach the robot, but their size is not their cost. The command change shows directly in the robot's motion after 5 controls and is still detectable after 60. Perturbation families that move the robot more tend to cost more success. For individual runs, however, the size of the early motion change is a poor predictor of failure: the AUROC, where 0.5 is chance, is 0.72 in $\pi_{0.5}$ and 0.64 in π_0 with all families pooled, and 0.62 and 0.55 within a family. Net success also hides lost and gained successes (Fig. 5; Sec. 5).

These results give research on VLA robustness two things. One is a common place to intervene: perturbations of different kinds act on the same few directions, and restoring them moves the commands. The other is a yardstick: any correction, ours included, should be judged by paired task outcomes against clean reruns.

2. Related Work

Visual robustness of VLA policies. LIBERO-Plus [5] and related evaluations [7, 21] report substantial success losses under visual, multi-modal, and adversarial perturbations. Robustness methods edit task-irrelevant image regions during deployment [8] or train for consistent actions under task-preserving input changes [7, 13, 16]. Task success, their common measure, shows that robustness is lost but not what a perturbation changes inside the policy, or whether larger action changes mean larger losses. We measure both.

Mechanistic analysis and internal interventions. Prior analyses trace representations and knock out attention [19], patch activations [14] to locate language grounding [22], and relate interventional visual attribution to generalization [24]. Others find features and directions that steer robot behavior [4, 6, 9, 20], or steer activations toward subspaces fitted on successful and failed rollouts, which transfer across tasks with similar failure modes [15]. We instead study the feature changes that visual perturbations cause, which any internal correction of visual robustness must act on. We fit directions to these changes on clean–perturbed pairs matched in state and policy noise, without outcome labels, hold out tasks and perturbation families together, and test the directions by

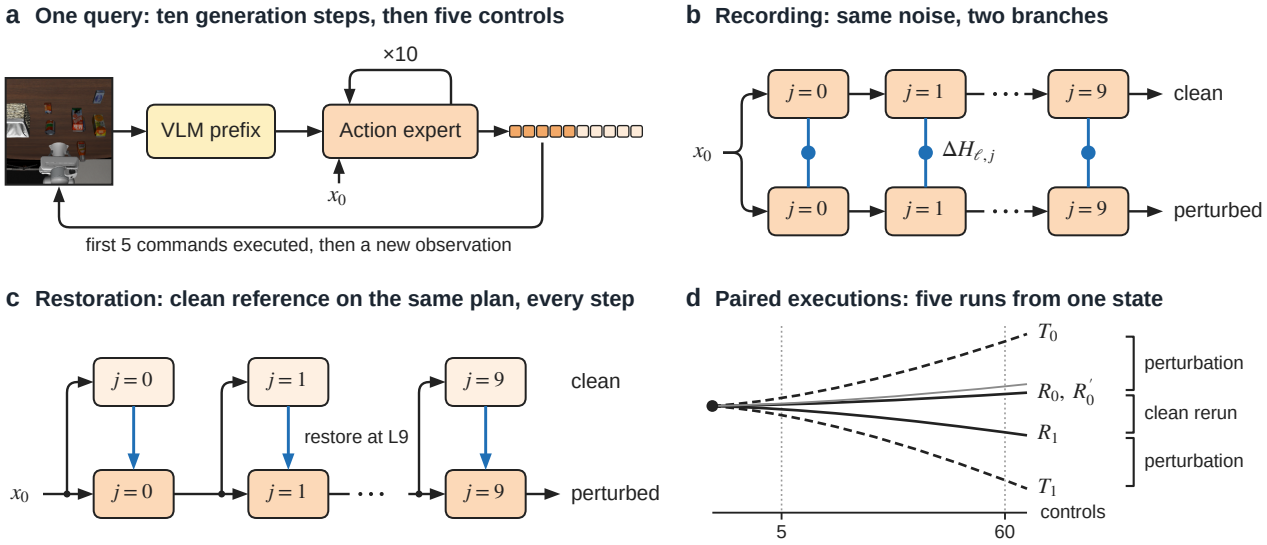


Figure 2. **How the policy runs, and what each study compares.** (a) One query: the prefix is computed once; the action expert refines the plan from policy noise x_0 over ten generation steps, and the first five actions (filled) are executed before the next observation. (b) Internal recordings: a clean and a perturbed branch start from the same x_0 and refine their plans separately; the response $\Delta H_{\ell,j}$ is their difference at generation step j and recorded layer ℓ. (c) Restoration: at every generation step of every query, the policy is also run on a clean observation of the perturbed run's current state, with the same partly refined plan (top row). The clean values along the 16 shared directions are copied into the perturbed run after L9, and generation continues with the perturbed prefix. (d) Paired executions: the distance between a perturbed run T_i and the clean run R_i with the same policy noise, after 5 and 60 controls, is compared with the distance between the two clean runs R_0 and R_1 (clean rerun); R_0' repeats R_0 with the same noise. Trajectories are schematic.

a subspace form of activation patching applied during the policy's own generation.

Evaluating robot behavior beyond task success. Similar success rates can accompany different smoothness, efficiency, and gripper behavior in successful trajectories [10]. Matched executions under individual and combined perturbations reveal opposing outcome transitions that cancel in aggregate success [18], and out-of-distribution inputs alone do not predict failure [25]. Run-to-run variation is a known hazard when comparing learned policies [1], and samples that a model update turns from correct to wrong are tracked as negative flips in classification [23]. We bring both to perturbed VLA executions: the variation between two clean runs that differ only in policy noise is the scale, and lost and gained successes are tracked under single perturbations.

3. Analysis Framework

All studies use frozen, LIBERO-finetuned $\pi_{0.5}$ [17] and π_0 [2] in the LIBERO simulator, under the four LIBERO-Plus visual perturbation families: Background, Camera, Light, and Sensor Noise. Table 1 lists the three studies.

Study	Comparison, timing, and data
Internal recordings ($\pi_{0.5}$)	Clean vs. perturbed features and commands at the first query: 404 pairs on 28 tasks (536 on all 40; Sec. 4.1)
Restoration ($\pi_{0.5}$)	Six runs per case: clean, perturbed, restoration, two controls, and the whole L9 state replaced; commands and final success: 96 Camera and 120 Sensor Noise cases, 1,296 rollouts on 31 held-out tasks (Sec. 4.2)
Paired executions (both policies)	Two clean, two perturbed, and one repeat run per setting; commands, motion after 5 and 60 controls, final success. Per policy: 268 settings on 40 tasks (primary: 202 on 28 tasks); 2,680 planned rollouts across both policies (Sec. 5)

Table 1. **Three paired studies, from features to task outcomes.** Setting: one task under one visual variant. Pair or case: one setting under one policy-noise draw; its runs share a start state.

3.1. How the Policy Turns an Observation into Commands

Each time the policy is called (a *query*), a vision-language network encodes the two camera images and the task instruction into context tokens, the *prefix* C (Fig. 2a). An *action expert*, a transformer with 18 blocks, then generates a plan:

a short sequence of future actions. Every block attends to the prefix, so the image enters at every layer.

The plan starts as random noise x_0, the *policy noise*, and is refined in ten *generation steps*:

$$x_{j+1} = x_j + \Delta t_j F_\theta(C, x_j, t_j), \quad j = 0, \ldots, 9, \quad (1)$$

where F_θ is the action expert's update at generation time t_j. Because the noise is random, two runs produce different plans even from identical observations. A plan holds ten actions for $\pi_{0.5}$ and fifty for π_0; each action commands a translation and a rotation of the end effector and a gripper movement. The robot executes the first five actions, takes a new observation, and queries again. We call each executed action a *control*, and the translations commanded by an episode's first five controls, 15 numbers, the *first five commands*.

To see what a perturbation changes inside the policy, we record $\pi_{0.5}$'s hidden states after blocks 1, 9, and 18 of the action expert (L1, L9, L18; Fig. 1a; Supplement, Sec. S1). With $H^c_{\ell,j}$ and $H^p_{\ell,j}$ the states after block ℓ at generation step j under the clean and the perturbed observation (Fig. 2b), the *response* to the perturbation is their difference:

$$\Delta H_{\ell,j} = H^p_{\ell,j} - H^c_{\ell,j}. \quad (2)$$

For $\pi_{0.5}$, $\Delta H_{\ell,j} \in \mathbb{R}^{10\times 1024}$: one row per planned action, 1024 channels.

3.2. What Is Compared

A *setting* is one task under one visual variant, for example one camera viewpoint. Every comparison runs a setting twice, clean and perturbed, from the same initial physical state and with the same policy noise, so that only the image differs. The two runs share a state only at the first query, where responses and commands are compared; motion is compared after 5 and 60 controls, and task success at the end of the episode.

Internal recordings and paired executions cover the same 268 settings on 40 tasks, in separate episodes. Primary comparisons use the 202 settings to which LIBERO-Plus assigns a difficulty level, on the 28 tasks that have all four families. The settings are a subset of LIBERO-Plus that we chose deliberately: recording internal states and running five executions per setting made the full benchmark impractical, and we left out settings whose perturbation we judged too large for a controlled comparison with the clean run, above all large camera displacements. The labelled settings span all five LIBERO-Plus difficulty levels. Success rates on this panel should not be compared with full-benchmark figures: for π_0 they are close under Background and Sensor Noise (85.7% and 78.6%, against 81.4% and 79.0%) but much higher under Camera (66.1% against 13.8%) [5]. Each setting is run under two draws of policy noise; one setting under one draw is a *pair* (a *case* in the restoration study), so the recordings give 404 pairs. The restoration study splits the 40 tasks into 9 used to develop it and 31 held out to test it (Sec. 4.2; Fig. 2c).

The paired executions run each policy five times per setting (Fig. 2d): two clean runs with different policy noise, two perturbed runs that each share the noise of one clean run, and a repeat of the first clean run, which checks that runs are reproducible. A perturbed run is always compared with the clean run that shares its noise. The two clean runs differ only in their policy noise, so their difference shows how much motion and outcomes vary without any perturbation. We use it as the reference for a perturbation's effect and call the second clean run a *clean rerun*.

All averages weight tasks equally, and intervals resample whole tasks. Of the 2,680 planned outcomes of the paired executions, 39 are missing (technical failures or failed validation); on the 28 common tasks they affect only Light. We keep them in the denominator and report bounds (Supplement, Secs. S1, S5, and S6).

4. Shared Response Directions and Their Effect on Commands

This section follows the first two links in $\pi_{0.5}$: what a perturbation changes in the features, and whether undoing that change moves the commands.

4.1. Different Perturbations Change Features along Shared Directions

Do different perturbations change the features in a common way? At each recorded layer we fit 16 directions to the responses by principal component analysis (PCA), with the policy frozen, and call them the *shared directions*. PCA finds the directions along which the fitted responses vary most, so a fair test must use responses it has not seen. Each fit therefore leaves out both the tasks and the perturbation family on which it is tested: directions tested on Camera responses, for example, are fitted without any Camera response and without the test tasks. Tasks that share a clean scene are left out together.

We then ask how much of an unseen response these directions retain. Responses are taken at the first generation step ($j = 0$) of the first query, where the clean and perturbed runs still share the same noisy plan and differ only in the image. With ΔH_i the response for the i-th planned action, μ_i its mean over the fitting data, and $B \in \mathbb{R}^{1024\times 16}$ the directions, the reconstruction is

$$\widehat{\Delta H}_i = \mu_i + (\Delta H_i - \mu_i) B B^\top. \quad (3)$$

We report the share of the response energy, its squared magnitude around μ, that the reconstruction retains (Supplement, Sec. S3). Two controls also use 16 directions: *clean-feature*

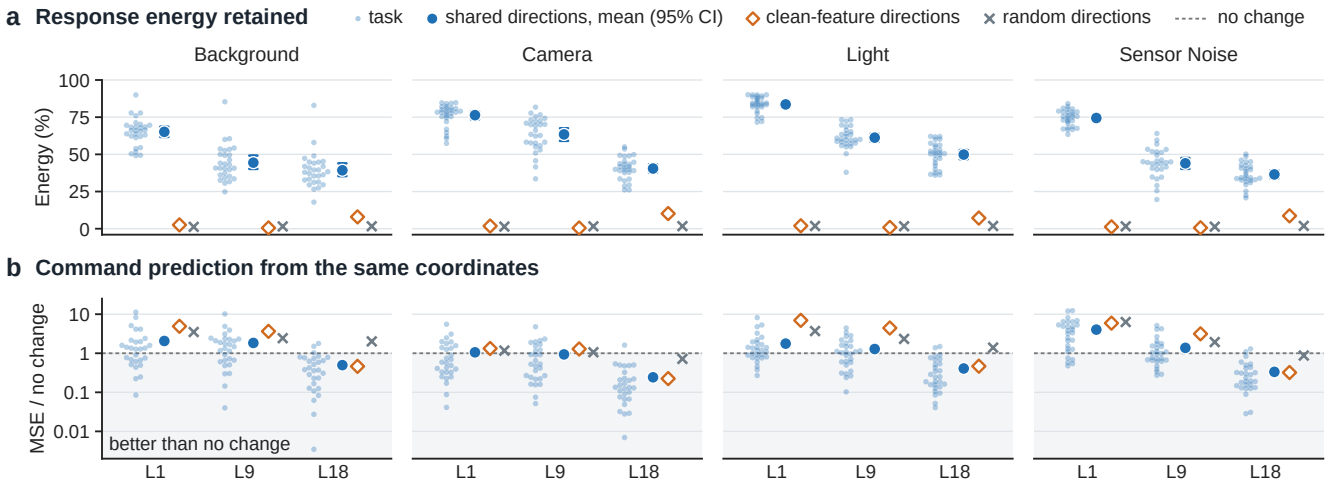


Figure 3. **Shared directions retain far more of an unseen response than the controls; the commands are predicted best at the last layer.** Both panels: 404 pairs on 28 tasks; each fit leaves out the test tasks and the test family. For the shared directions, small dots are tasks and large dots their means; diamonds and crosses are the two controls. (a) Share of the response energy that 16 directions retain; the means carry pointwise 95% intervals. (b) Error of a linear readout that predicts the change in the first five commands from the same 16 coordinates, divided by the family-mean error of predicting that the commands do not change; large markers are ratios of task means. Below 1 (shaded), the readout beats predicting no change. Paired comparisons: Supplement, Secs. S3.4 and S13.

directions, fitted to the policy's clean features, which test whether responses simply follow the way features ordinarily vary, and *random directions*, which show what chance alone gives.

The shared directions retain 36.5–83.6% of the response energy, depending on layer and family. Clean-feature directions retain at most 10.2%, and random directions about 1.6% (Fig. 3a). These are shares of the energy around μ; of the raw response energy, the same procedure retains 16.3–75.0% on the full 40-task panel (Supplement, Table S5). The directions were never fitted to the tasks or the perturbation family on which they are tested, so the match cannot come from fitting to the test data: different perturbations change the features along common directions. The result holds for other numbers of directions, in π_0, and at later observations of an episode (Supplement, Secs. S3, S14, and S8.1).

From responses to commands. A linear readout of the 16 coordinates predicts the change in the first five commands best at L18, the last block before the output, and poorly at L1 and L9 (Fig. 3b; Supplement, Secs. S3.4 and S13). A readout tests what can be read from these coordinates; restoring them inside the policy tests what they change. We therefore test the directions directly, by intervening.

4.2. Restoring Shared Components Reduces Command Error

If the shared directions matter for the commands, undoing the perturbation along them should move the commands back toward those of the clean run. *Restoration* does this at L9. At every generation step of every query, it computes what the L9 state would be under a clean observation of the same physical state, and copies the clean values along the 16 shared directions into the perturbed run (Fig. 2c). With H^c and H^p the clean and perturbed L9 states,

$$H' = H^p - (H^p - H^c)BB^\top. \tag{4}$$

Everything else stays perturbed: the rest of the L9 state, and the image that blocks 10–18 still read. This is activation patching [14] restricted to the shared directions. It needs a clean observation of the same state, so it is a controlled test and not a method one could deploy. We fixed L9, the middle layer, in advance, as a compromise between L1, which retains the most response, and L18, which predicts commands best.

We test restoration on 216 cases (96 Camera, 120 Sensor Noise) from 31 tasks, none of them among the nine used to develop it. Each case is run six times from the same start state with the same policy noise (Table 2): clean; perturbed; perturbed with restoration; two controls that restore 16 clean-feature or random directions; and a run that replaces the whole L9 state, which shows the most that restoring this layer can do. Task success is the primary endpoint, fixed in advance. The second endpoint is the *command error*: the distance between a run's first five commands and those of the clean run. Directions are again fitted without the test tasks and family (Supplement, Sec. S12).

Command error falls. Restoration lowers the command error by 24% for Camera (0.409 to 0.312) and by 5% for Sensor Noise (0.209 to 0.199), while the two controls leave

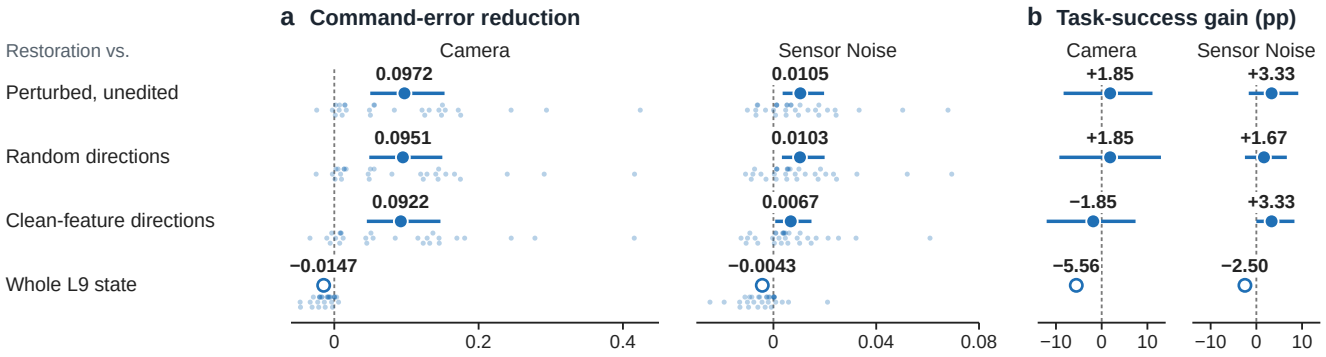


Figure 4. **Restoration against the unedited perturbed run, the two controls, and replacing the whole L9 state.** Each row compares restoration with another run of the same cases (31 held-out tasks; Camera: 27 tasks, 96 cases; Sensor Noise: 30 tasks, 120 cases). Positive values favor restoration; negative values favor the other run, as for the whole L9 state on both measures. (a) Command error of the other run minus that of restoration: mean with simultaneous 95% interval; small dots are tasks. Restoration lowers the command error of the unedited run in 25 of 27 and 24 of 30 tasks, and reduces it more than both controls, which also write less (edit size, Table 2). (b) Primary endpoint: success of restoration minus that of the other run, in percentage points, with intervals as in (a). The random-direction run under Camera and the clean-feature run under Sensor Noise succeed as often as the unedited run (Table 2), so their rows repeat its means. The whole-L9 run was not among the six prespecified comparisons, so it has no interval (open marks). Axes differ between families.

	Camera			Sensor Noise		
Run	error	succ.	edit	error	succ.	edit
Perturbed, unedited	0.409	73.15	–	0.209	83.33	–
Random directions	0.407	73.15	9.7	0.209	85.00	9.6
Clean-feature directions	0.404	76.85	7.8	0.205	83.33	7.7
Restoration	0.312	75.00	46.2	0.199	86.67	51.5
Whole L9 state	0.297	80.56	74.2	0.194	89.17	72.8
Clean	0	98.15	–	0	97.50	–

Table 2. **The six runs of the restoration test.** Command error (distance of the first five commands from those of the clean run), task success (%), and edit size (summed change written into the L9 state over an episode, in thousands), averaged with equal weight per task over 96 Camera and 120 Sensor Noise cases. Intervals for the comparisons: Figs. 1b and 4; Supplement, Tables S24–S26.

it almost unchanged (Table 2; Figs. 1b and 4a). The error falls in 25 of 27 Camera tasks and 24 of 30 Sensor Noise tasks, and the intervals for all comparisons, with the unedited run and with both controls, exclude zero (Supplement, Sec. S12.4). The shared directions at L9 therefore carry part of the perturbation's effect on the commands.

The shared directions do most of what this layer can do. Replacing the whole L9 state removes 27% and 7% of the command error; the rest enters through the later blocks, which still read the perturbed image. The 16 shared directions, out of 1024 channels, reach 86.9% and 70.7% of that (Supplement, Tables S25–S26).

Controls and edit size. The two controls write less than restoration: the shared directions, fitted to clean–perturbed differences, copy five to seven times as much (edit size in Table 2). Per unit of edit, restoration removes 1.4 times as much command error as replacing the whole state under Camera, and as much under Sensor Noise. With edit size matched in development, it reduces command error more than clean-feature directions under Camera but not under Sensor Noise (Supplement, Sec. S12.5). A second test needs no clean observation: on original LIBERO (6,048 planned rollouts), removing the shared components at L9 lowers the response at L18 by 8.8% and 7.6% under a camera and a lighting change, against at most 0.3% for size-matched random directions (Supplement, Sec. S11).

Task success is also higher. After restoration, success is higher by 1.85 points for Camera and by 3.33 for Sensor Noise (Table 2; simultaneous 95% intervals $[-8.33, 11.11]$ and $[-1.67, 9.17]$, Fig. 4b; Supplement, Table S24). Replacing the whole L9 state raises success by 7.41 and 5.83 points (post hoc pointwise intervals $[0.93, 14.81]$ and $[0.00, 12.50]$; Supplement, Sec. S12.4): this layer matters for the outcome, and the 16 directions deliver part of that gain. Section 5 turns to how command changes relate to motion and success.

5. From Action Commands to Motion and Task Outcomes

This section follows the last two links. Do changed commands move the robot, and does the size of that motion tell how much success is lost?

5.1. From Initial Commands to Motion

Section 4.2 judged restoration by its effect on the first five commands, so we first check that changes in these commands reach the robot. For every clean–perturbed pair, on all 40 tasks, we compare how much the first five commands

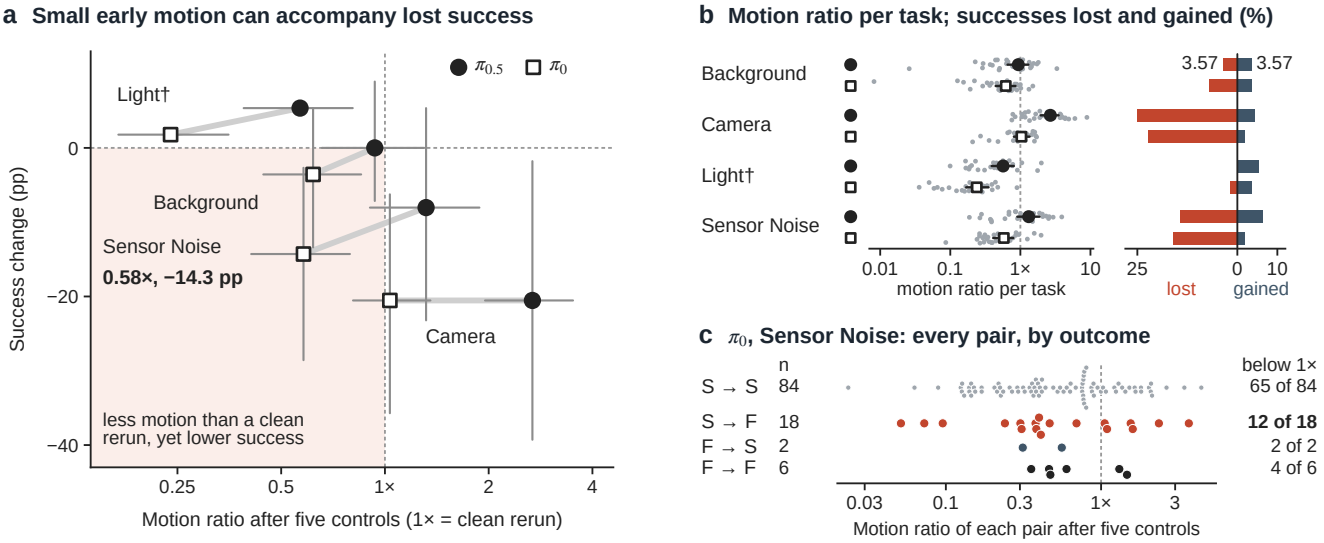


Figure 5. **A small motion change can cost success, and net success hides lost and gained successes.** Unedited policies, 28 common tasks. Motion ratio: distance between a perturbed run and its clean run after five controls, relative to the distance between two clean runs (1×: as far as a clean rerun). (a) Motion ratio of each family (95% interval) against its change in success (98.75% interval, i.e., 95% Bonferroni-adjusted over four families); grey bands join the two policies of a family. In the shaded region a perturbation moves the robot less than a clean rerun and still lowers success: under Sensor Noise, π_0 moves 0.58× and loses 14.3 percentage points. (b) Left: motion ratio of every task (grey dots) and each family (markers). Right: successes lost and gained; for $\pi_{0.5}$ under Background both are 3.57%. (c) All 110 π_0 Sensor Noise pairs by outcome (S, success; F, failure; clean→perturbed), each relative to its own setting's clean rerun; the right column counts pairs below 1×. Pairs that lose a success (S→F) cover a similar range to pairs that keep it (S→S). † For Light, some outcomes were not recorded: its marks use recorded outcomes only and carry no success interval, and its motion covers 43/48 settings on 26 tasks ($\pi_{0.5}$) and 44/48 on 27 (π_0). Bounds: Supplement, Sec. S6.

	Motion	Success		Perturbation			Rerun
Family	ratio	clean	pert.	lost	gained	all	all
$\pi_{0.5}$							
Background	0.93	94.6	94.6	3.6	3.6	7.1	3.6
Camera	2.68	94.6	74.1	25.0	4.5	29.5	3.6
Light†	0.57	91.1	–	–	–	–	–
Sensor Noise	1.32	92.9	84.8	14.3	6.3	20.5	7.1
π_0							
Background	0.62	89.3	85.7	7.1	3.6	10.7	7.1
Camera	1.03	86.6	66.1	22.3	1.8	24.1	5.4
Light†	0.24	87.5	–	–	–	–	–
Sensor Noise	0.58	92.9	78.6	16.1	1.8	17.9	0.0

Table 3. **How far each perturbation moves the robot, and what it does to task outcomes.** Motion ratio: Sec. 5.2. The other columns are percentages over the 28 common tasks: success of the clean and the perturbed run; outcomes that succeed when clean and fail when perturbed (lost), the reverse (gained), and both (all); and outcomes that differ between the two clean runs (rerun). †Some Light outcomes were not recorded, so no rates are given; Fig. 5 shows the recorded ones (bounds: Supplement, Sec. S6).

changed with how far apart the two end effectors are after 5 controls and after 60.

Early on, the two agree almost perfectly: the rank correlation is 0.95 in $\pi_{0.5}$ and 0.97 in π_0. This is expected, since the first five controls simply execute those commands. After 60 controls, eleven replans later, the correlation is 0.47 and 0.41: weaker, but still clearly present (Supplement, Sec. S5.1). A perturbation's first effect on the commands thus remains visible in the robot's motion well into the episode.

The command change is also systematic: with a different draw of policy noise, the same perturbation changes the commands in nearly the same way (agreement 0.88–0.92 in $\pi_{0.5}$ and 0.80–0.89 in π_0, where 1 is identical; Supplement, Sec. S7). The first five commands are therefore a repeatable readout of what a perturbation does to the robot.

5.2. Motion Size and Task Outcomes

How large is a motion change? Two clean runs already differ, because each draws its own policy noise. We use that difference as the yardstick (Sec. 3.2). The *motion ratio* is the distance between a perturbed run and its clean run, divided by the distance between two clean runs, after five controls. Below 1, the perturbation moves the robot less than a clean rerun does. Table 3 lists it with the outcomes.

A small motion change can cost success. Take π_0 under Sensor Noise. After five controls, the perturbed arm

is 3.5 mm from where the clean run put it, while a clean rerun lands 6.1 mm away. Yet in these settings a rerun never changes whether the task succeeds, whereas the perturbation costs 14.3 points of success (interval $[-28.6, -2.7]$; Fig. 5a).

Motion size is a poor predictor of lost success. Across families, larger motion changes mostly come with larger losses: Camera moves the robot most and costs the most success in both policies (Table 3). For individual runs the link is weak. We rank the pairs by their motion change and ask how well the ranking separates pairs that lose a success from pairs that keep it. This is the AUROC (0.5 is chance, 1 is perfect), computed post hoc. With all families pooled it is 0.72 in $\pi_{0.5}$ and 0.64 in π_0; after 60 controls it is 0.60 and 0.68; and raw distance or the size of the command change gives nearly the same values as the ratio. The pooled values mostly reflect differences between families: within a family the AUROC drops to 0.62 and 0.55, and for π_0 under Sensor Noise to 0.49 (Fig. 5c; Supplement, Secs. S5 and S6).

Three further comparisons show the same gap. Under Camera, π_0 moves about as far as a clean rerun ($1.03\times$) and loses 20.5 points (Table 3). In a ten-condition study of $\pi_{0.5}$ on original LIBERO, a condition's mean command change ranks its mean success change (rank correlation -0.90), but across the 400 task–condition cells the correlation is -0.075 (Supplement, Sec. S9.1). And removing a headlight changes the first five commands 5.1 times as much as a dimming matched in image statistics, yet both succeed in 77 of 80 cases (Supplement, Sec. S9.2).

Net success hides lost and gained successes. Under a perturbation some clean successes fail and some clean failures succeed; the net change shows only their difference. Under Sensor Noise, $\pi_{0.5}$ gains nearly half as many successes as it loses, so its net loss of 8.0 points understates the 20.5% of outcomes that change (Table 3; Fig. 5b). A clean rerun alone changes up to 7.1% of outcomes (10.7% for $\pi_{0.5}$ under Light), so changes must be read against it: lost successes exceed the rerun level under Camera and Sensor Noise in both policies, while gained successes and the changes under Background do not (Supplement, Sec. S6.1). Paired runs under combined perturbations show similar cancellations [18]; the rerun comparison adds the baseline for reading them.

Sensitivity is therefore not fragility: a perturbation can move the robot less than a clean rerun and still cost success, motion size hardly tells which runs fail, and robustness has to be judged by paired outcomes against clean reruns.

6. Discussion and Limitations

What restoration achieves. With 16 directions at one layer, restoration reduces command error in most tasks, and the paired outcomes of the test follow a consistent pattern (Supplement, Sec. S12.4). Failures turned into successes grow with how much of the clean state is copied: 3 to 6 under the controls, 7 under restoration, and 10 when the whole L9 state is replaced. Restoration itself gains 7 successes and loses 5 under Camera, and gains 7 and loses 3 under Sensor Noise. Every edited run, including the random-direction control, also turns 3 to 5 of the 96 Camera or 120 Sensor Noise cases from success into failure, much as a clean rerun does.

Limitations and next steps. Sensitivity is measured at the start of an episode; later, contact-critical steps are left to future work. Restoration is tested in $\pi_{0.5}$ at one layer, with 16 directions, on two of the four families and 216 cases, and it needs a clean observation of the same state, which deployment lacks. The shared directions also carry the response to Background, which costs $\pi_{0.5}$ no net success (39–65% of its response energy lies along directions fitted to the other families), so they describe how the policy responds to a visual change, harmful or not. All experiments are simulated, use two related policies, and use settings chosen to keep perturbations moderate. Each limit names a next step, from tracking sensitivity through an episode to estimating clean values without a clean observation.

7. Conclusion

Sensitivity is not fragility: across two VLA policies and four visual perturbation families, how strongly a policy responds to a perturbation early in an episode is a poor guide to how much task success it loses. Inside $\pi_{0.5}$, different perturbations change features along 16 shared directions per layer, and copying clean values back along them reduces command error. In paired executions, a perturbation can move the robot less than a clean rerun and still cost success, motion size is a poor predictor of which runs fail, and net success hides changes in both directions. These results give later work a foundation: a common internal target, the shared directions, and a yardstick, paired outcomes against clean reruns, by which to judge any correction, ours included.

LLM Usage Statement

The authors conceived the research ideas and core experiments and made the final scientific decisions. LLM tools—ChatGPT and Codex (OpenAI), Claude (Anthropic), and Kimi (Moonshot AI)—assisted with implementing and debugging experimental and analysis code, refining experimental designs, preparing figures, and drafting and editing the manuscript and supplementary material. The authors checked the assisted work and take responsibility for the experiments, data, analyses, citations, and scientific conclusions.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021. 3

[2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 2, 3

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818v1*, 2023. 2

[4] Hugo Buurmeijer, Carmen Amo Alonso, Aiden Swann, and Marco Pavone. Observing and controlling features in vision-language-action models. *arXiv preprint arXiv:2603.05487v2*, 2026. 2

[5] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: A progressive robustness benchmark for visual-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 38574–38583, 2026. 2, 4

[6] Bryce Grant, Xijia Zhao, and Peng Wang. Not all features are created equal: A mechanistic study of vision-language-action models. arXiv preprint arXiv:2603.19233v1, 2026. 2

[7] Jianing Guo, Zhenhong Wu, Chang Tu, Yiyao Ma, Xiangqi Kong, Zhiqian Liu, Jiaming Ji, Shuning Zhang, Yuanpei Chen, Kai Chen, Qi Dou, Yaodong Yang, Xianglong Liu, Huijie Zhao, Weifeng Lv, and Simin Li. RobustVLA: On robustness of vision-language-action model against multi-modal perturbations. In *International Conference on Learning Representations*, 2026. Author manuscript arXiv:2510.00037v6. 2

[8] Asher J. Hancock, Allen Z. Ren, and Anirudha Majumdar. Run-time observation interventions make vision-language-action models more visually robust. *arXiv preprint arXiv:2410.01971*, 2024. 2

[9] Bear Häon, Kaylene Stocking, Ian Chuang, and Claire Tomlin. Mechanistic interpretability for steering vision-language-action models. In *Conference on Robot Learning*, 2025. 2

[10] Sophie Higham, Riccardo Andrea Izzo, Matteo Matteucci, and Alessandro Suglia. Is success all you need? investigating the impact of input perturbations on vla behaviour in tabletop manipulation tasks. *arXiv preprint arXiv:2610.01351v1*, 2026. 3

[11] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246v3*, 2024. 2

[12] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, pages 44776–44791, 2023. Datasets and Benchmarks Track. Author manuscript arXiv:2306.03310v2. 2

[13] Jingzhou Luo, Yifan Wen, Yongjie Bai, Xinshuai Song, Yang Liu, and Liang Lin. RoVLA: Multi-consistency constraints for robust vision-language-action models. *arXiv preprint arXiv:2605.19678v1*, 2026. 2

[14] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, pages 17359–17372, 2022. 2, 5

[15] Miranda Muqing Miao, Subin Kim, Brandon Yang, and Lyle Ungar. Contrastive conceptor activation steering (COAST): Unlocking vision-language-action models through hidden states, 2026. arXiv:2605.17144v1. 2

[16] Yuanfang Peng, Jingjing Fu, Chuheng Zhang, Li Zhao, Jiang Bian, Mingyu Liu, Ling Zhang, Jun Zhang, and Rui Wang. What to ignore, what to react: Visually robust RL fine-tuning of VLA models. *arXiv preprint arXiv:2605.13105v1*, 2026. 2

[17] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz,

James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 2, 3

[18] Hiroki Sawada and Shunichi Kasahara. Beyond single-axis testing: Paired evaluation of compound robustness in vision-language-action policies. *arXiv preprint arXiv:2609.15940v2*, 2026. 3, 8

[19] Haoyuan Shi, Xiancong Ren, Yingji Zhang, Qinfan Zhang, Jiayu Hu, Haozhe Shan, Han Dong, Jinpeng Lu, Yinda Chen, Yi Zhang, Yong Dai, and Xiaozhu Ju. VLA-Trace: Diagnosing vision-language-action models through representation and behavior tracing. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 47280–47315. Association for Computational Linguistics, 2026. 2

[20] Aiden Swann, Lachlain McGranahan, Hugo Buurmeijer, Monroe Kennedy, III, and Mac Schwager. Sparse autoencoders reveal interpretable and steerable features in VLA models. *arXiv preprint arXiv:2603.19183v2*, 2026. 2

[21] Taowen Wang, Cheng Han, James Chenhao Liang, Wenhao Yang, Dongfang Liu, Luna Xinyu Zhang, Qifan Wang, Jiebo Luo, and Ruixiang Tang. Exploring the adversarial vulnerabilities of vision-language-action models in robotics. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6948–6958, 2025. 2

[22] Theodor Wulff and Angelo Cangelosi. Do vision-language-action models understand instructions? a mechanistic interpretability study on language grounding. *arXiv preprint arXiv:2610.10178v1*, 2026. 2

[23] Sijie Yan, Yuanjun Xiong, Kaustav Kundu, Shuo Yang, Siqi Deng, Meng Wang, Wei Xia, and Stefano Soatto. Positive-congruent training: Towards regression-free model updates. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14299–14308, 2021. 3

[24] Hanxin Zhang, Mingshuo Xu, Abdulqader Dhafer, Shigang Yue, Hongbiao Dong, and Zhou Daniel Hao. Embodied interpretability: Linking causal understanding to generalization in vision-language-action models. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 159526–159549. PMLR, 2026. Author manuscript arXiv:2605.00321v2. 2

[25] Kaiwen Zhu, Dongfang Liu, and Liangkai Liu. VLA-Scope: Shift-aware failure prediction for vision-language-action models. *arXiv preprint arXiv:2609.21246v1*, 2026. 3